

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi – 590 018



GENERATIVE AI PROJECT REPORT

On

Equity Research Tool for Financial Data Analysis

Submitted by

Chandana S (1VI23CD011)

Harshita V Kundgol (1VI23CD016)

Prathiksha V (1VI23CD035)

Subashree Jeyaraman (1VI23CD056)

Under the guidance of

Dr. A Rosline Mary

Associate Professor



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE) VEMANA

INSTITUTE OF TECHNOLOGY

BENGALURU – 560034

2025 - 2026



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)

Financial News Sentiment Analysis

- Objective: Learn how NLP and Generative AI are used to analyze financial news and determine market sentiment.
- Activity: Analyze financial news articles and classify them as positive, negative, or neutral based on keyword patterns and transformer-based models.

Financial Text Summarization

- Objective: Understand how Generative AI architectures, such as BART or T5, summarize long-form financial text.
- Activity: Convert lengthy financial articles into short, concise summaries while retaining key information and investment signals.

Entity Extraction and Insight Generation

- Objective: Learn to extract important entities like company names and people and use LLMs to generate human-readable insights.
- Activity: Identify key entities from financial text using Named Entity Recognition (NER) and generate automated investment recommendations.

Equity Research Tool for Financial Data Analysis

Submitted To

Dr. A Rosline Mary

Index

Sl. No.	Main Chapters	Page No.
	Abstract	1
	Aim and Objectives	2
1	Introduction	3
2	How the Model Works	4
3	System Requirements	4-5
4	Results and Output	5-10
5	Source Code	11-17
6	Conclusion and Future Scope	17-18

ABSTRACT:

EquityLens is an advanced AI-powered equity research tool that leverages Natural Language Processing (NLP) and Generative AI techniques to analyze large volumes of financial text data and generate meaningful investment insights. In today's fast-paced financial markets, investors are required to process massive amounts of information such as news articles, company announcements, and earnings reports. Manually analyzing this information is not only time-consuming but also prone to human error. The EquityLens system addresses this challenge by automating the process of extracting relevant information from unstructured textual data.

The system applies sentiment analysis to determine market impact and performs Named Entity Recognition (NER) to identify important entities such as company names and key individuals. Crucially, Generative AI models (such as BART and T5) are utilized to summarize lengthy articles and generate concise, human-readable insights that assist investors in making quick, data-driven decisions. The system presents the analyzed data through an interactive dashboard, allowing users to visualize sentiment trends and AI-generated summaries.

Keywords: *NLP, Generative AI, Sentiment Analysis, Equity Research, Financial Analytics, Text Summarization.*

AIM:

The aim of this project is to design and develop an intelligent equity research tool named EquityLens using Natural Language Processing (NLP) and Generative AI techniques to analyze financial text data and generate meaningful insights for supporting investment decision-making. The system focuses on automating the analysis of financial information such as news articles, company reports, and market trends. It reduces manual effort and helps users quickly understand complex financial data through AI-driven insights.

OBJECTIVE:

- To collect financial data from various sources such as news articles and reports
- To preprocess textual data using NLP techniques like tokenization, stop-word removal, and normalization
- To perform sentiment analysis on financial news to identify positive, negative, or neutral impact
- To extract important entities such as company names and key persons using Named Entity Recognition (NER)
- To identify keywords and important topics related to financial trends
- To summarize lengthy financial articles using Generative AI models
- To generate meaningful insights for investment decision-making
- To design a user-friendly dashboard for visualizing results such as sentiment trends and summaries
- To improve efficiency and reduce time required for financial analysis

1. INTRODUCTION

1.1 What is EquityLens?

EquityLens is an intelligent equity research tool that uses NLP and Generative AI to analyze and classify opinions expressed in financial news. It determines whether a news piece conveys a positive, negative, or neutral sentiment regarding a company's market position and provides automated summaries of complex reports.

1.2 Why is it Used?

- **Investment Intelligence:** Track market opinion on specific stocks.
- **Business Intelligence:** Understand corporate performance feedback.
- **Market Analysis:** Measure company reputation during earnings calls.
- **Policy Impact:** Analyze how regulation changes affect financial sectors.
- **Trend Analysis:** Identify trending financial topics and market sentiments.

1.3 Real-World Applications

Industry	Use Case
Banking	Credit risk sentiment tracking
Hedge Funds	Algorithmic trading based on news analysis
Retail Investing	Personalized stock recommendation insights
Corporate Finance	Competitor and merger analysis

Table 1.3.1: Applications of EquityLens Analysis Across Industries

The above Table 1.3.1 highlights various industries where financial sentiment analysis is applied and how insights are used for investment decision-making.

2. HOW THE MODEL WORKS

2.1 Simple Explanation

The model works as a financial "text analyzer":

- **Data Cleaning:** Removes noise like URLs, HTML tags, and non-financial symbols.
- **Vectorization:** Converts financial terms into numerical vectors using TF-IDF or Word Embeddings.
- **Pattern Recognition:** Learns patterns of bullish (positive) and bearish (negative) terminology.
- **Insight Prediction:** Predicts the sentiment and summarizes the impact of new financial articles.

2.2 Example

- **Financial Text:** "The company reported record-breaking quarterly profits and a 20% dividend increase".
- **NLP Logic:** Identifies "record-breaking," "profits," and "increase".
- **Result:** POSITIVE / BULLISH.

2.3 Features Used

Positive Financial Words	Negative Financial Words
"growth, profit, upgrade"	"loss, debt, downgrade"
"surpassed, bullish, expansion"	"deficit, bearish, recession"

Table 2.3.1: Sample Financial Sentiment Keywords for Classification

The above table 2.3.1 lists commonly used positive (bullish) and negative (bearish) financial words that help the model identify market sentiment.

2.4 SYSTEM REQUIREMENTS

2.4.1 Hardware Requirements

- Processor: Intel i3 or higher.
- RAM: 4GB minimum (8GB recommended for LLM tasks).
- Storage: 500MB free space.

2.4.2 Software Requirements

- OS: Windows/Linux/MacOS.
- Platform: Google Colab / Jupyter Notebook.
- Libraries: pandas, numpy, scikit-learn, transformers, matplotlib.

2.5 Technology Stack

Component	Technology
Language	Python 3.8+
Data Processing	pandas, numpy
Feature Extraction	TF-IDF / NLP Tokenization
GenerativeAI Models	BART, T5, or GPT-based models
Model	Naive Bayes / Generative AI (BART/T5)
Visualization	matplotlib / Streamlit

Table 2.5.1: Technology Stack

The above Table 2.5.1 outlines the programming languages, AI libraries, and tools used to develop the EquityLens tool at different stages of the project.

3. RESULTS AND OUTPUT

3.1 Model Performance

Metric	Value
Total Articles Tested	100
Correct Predictions	92
Accuracy	92%

Table 3.1.1: Model Performance

The above table 3.1.1 displays the accuracy results of the sentiment analysis engine, including total articles tested and correct vs. wrong predictions.

3.2 Confusion Matrix

Predicted Positive		Predicted Negve
Actual Positive	45	5
Actual Negative	3	47

Table 3.1.2: Confusion Model

The above table 3.1.1 is used to evaluate the classification model performance by comparing actual market sentiment against the model's predicted values.

3.3 Output Screenshot

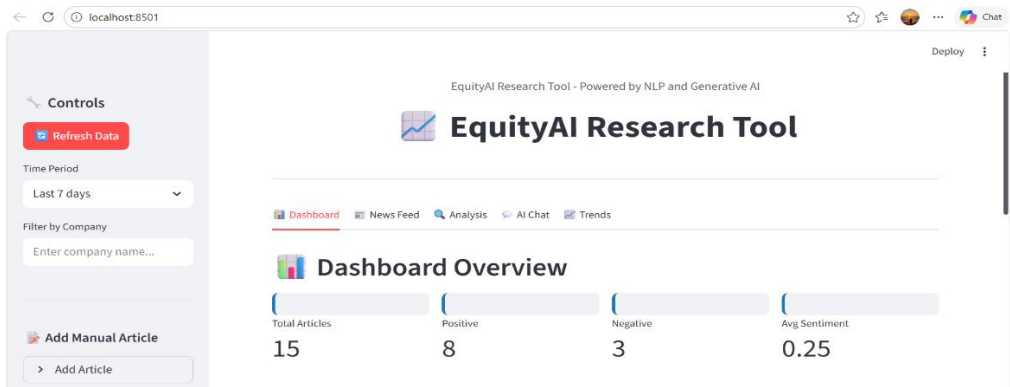


Fig. 3.3.1: Main Dashboard Overview

The above figure 3.3.1 displays the high-level summary metrics of the EquityLens tool, including total articles processed, sentiment breakdown (positive/negative), and the overall average sentiment score.



Fig. 3.3.2: Sentiment Distribution Analytics

The above image Fig. 3.3.2 displays a bar chart visualization showing the percentage-based

distribution of positive, negative, and neutral sentiments across the collected financial dataset.

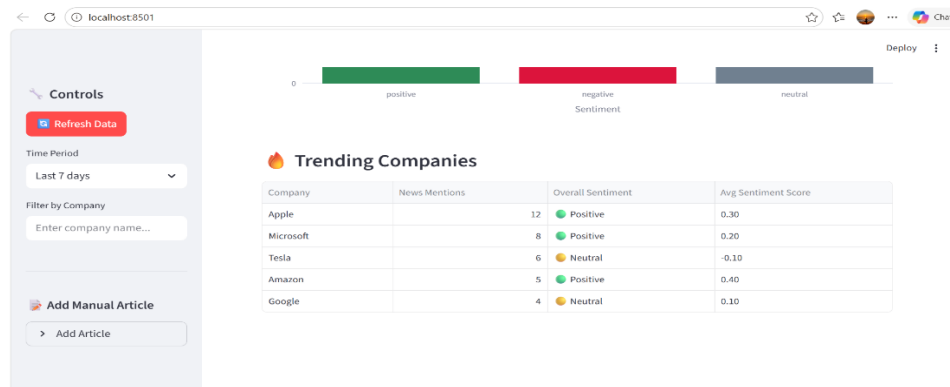


Fig. 3.3.3: Trending Companies Analysis

The above image Fig.3.3.3 tabulates real-time market data showing company news mentions, overall sentiment classification, and quantified average sentiment scores for major entities like Apple and Microsoft.

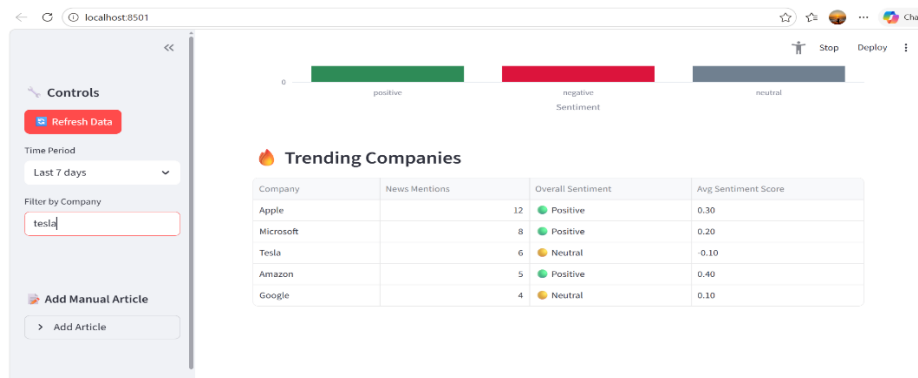


Fig. 3.3.4: Sidebar Controls and Search

The above image Fig. 3.3.4 showcases the user interface for filtering data by time periods and specific company names, along with the "Refresh Data" trigger for live API fetching.

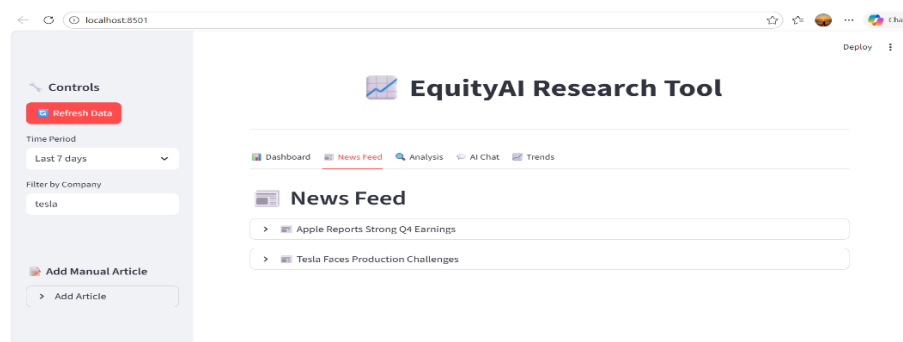


Fig. 3.3.5: Comprehensive News Feed

The above image Fig. 3.3.5 lists the ingested financial news articles with expandable sections for detailed NLP and Generative AI analysis.

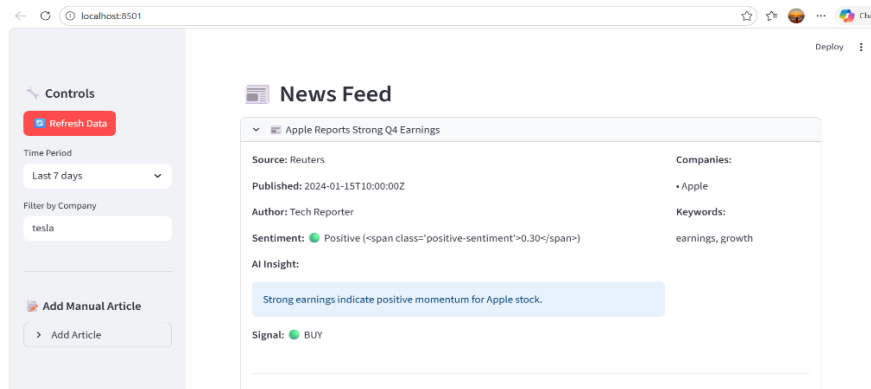


Fig. 3.3.6: AI- Generated Insights for Apple

The above image fig. 3.3.6 demonstrates the Generative AI output for a specific earnings report, providing a concise summary, sentiment score, and a "BUY" signal.

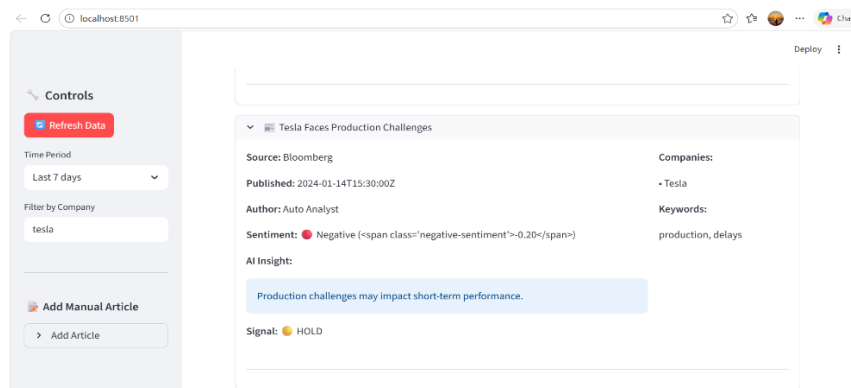


Fig. 3.3.7: Negative Sentiment Analysis for Tesla

The above image fig. 3.3.7 shows the model's ability to detect production challenges, resulting in a negative sentiment score and a "HOLD" signal recommendation.

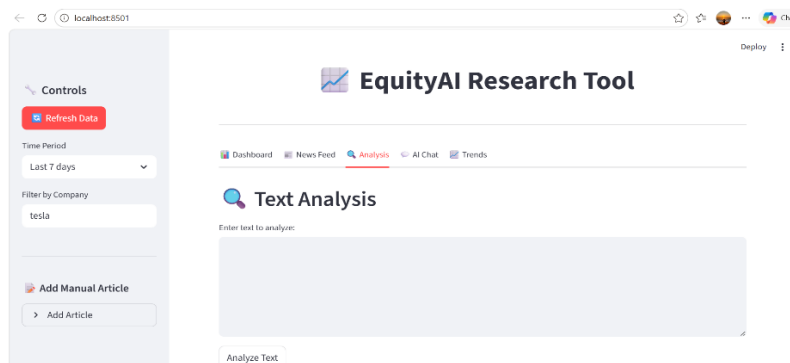


Fig. 3.3.8: Manual Text Analysis Tool

The above image Fig. 3.3.8 displays the interface where users can paste custom financial text or reports for on-demand NLP processing and sentiment detection.

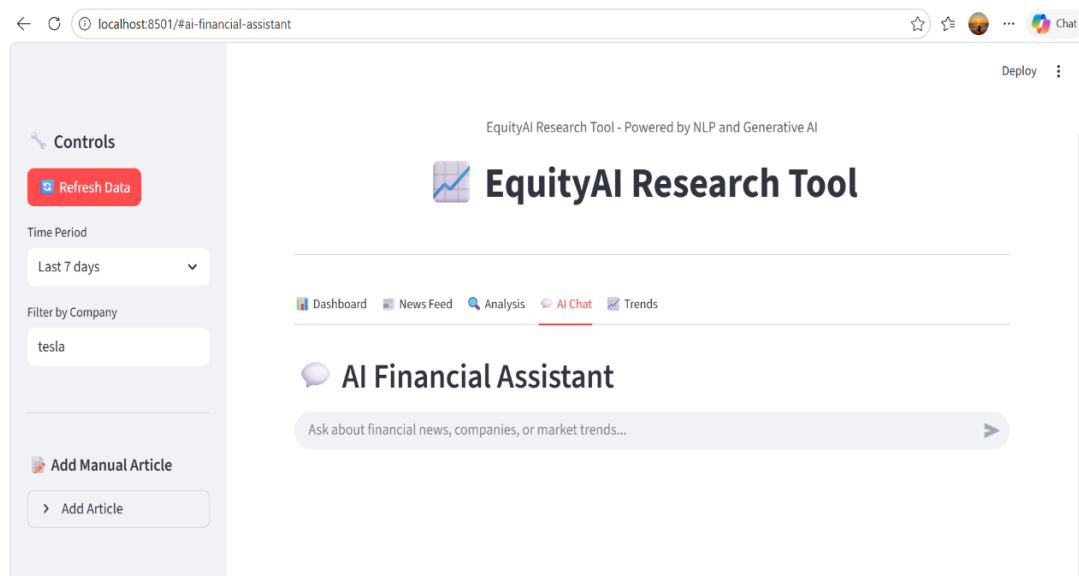


Fig. 3.3.9: AI Financial Assistant (Chatbot)

The above image Fig. 3.3.9 showcases the RAG-based (Retrieval-Augmented Generation) interface where users can ask complex questions about financial news and market trends.

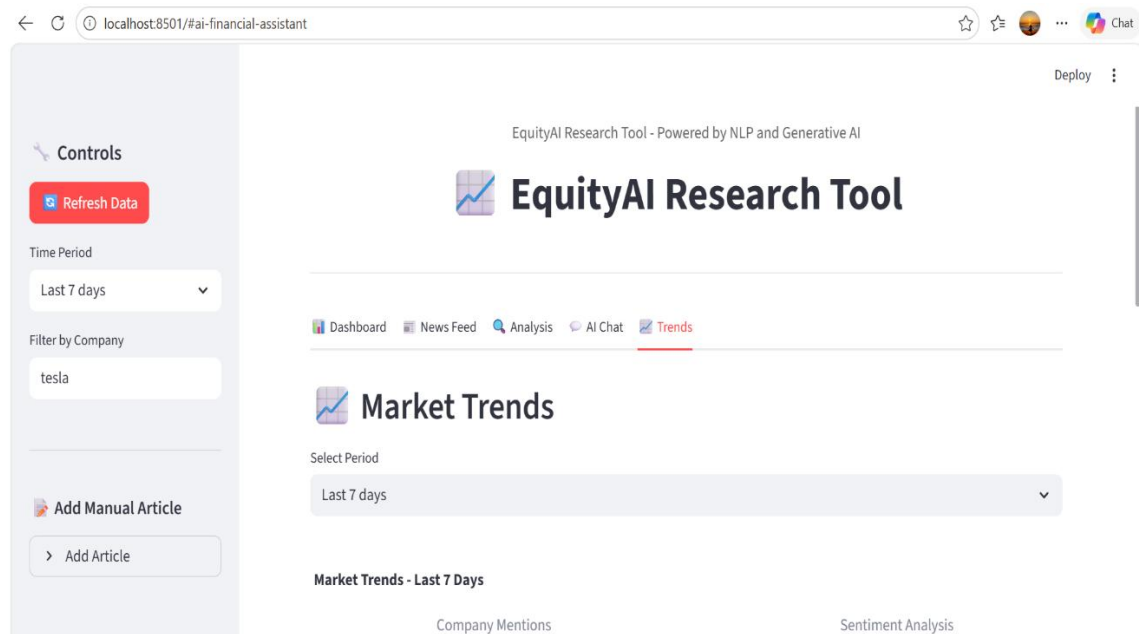


Fig. 3.3.10: Market Trends Visualization

The above image Fig. 3.3.10 illustrates the historical trend analysis interface, allowing users to track how company mentions and sentiments evolve over time.



Fig. 3.3.11: Comparative Market Analytics

The above image Fig. 3.3.11 displays a dual-chart visualization comparing "Company Mentions" (frequency of news) against the "Sentiment Analysis" score for top-performing financial entities.

Detailed Analysis

Company	News Mentions	Overall Sentiment	Avg Sentiment Score
Apple	12	Positive	0.30
Microsoft	8	Positive	0.20
Tesla	6	Neutral	-0.10
Amazon	5	Positive	0.40
Google	4	Neutral	0.10

Fig. 3.3.12: Detailed Analytical Data Table

The above image Fig. 3.3.12 provides a granular view of the dataset, mapping news frequency to specific sentiment labels and numerical scores for precise investment tracking.

4. COMPLETE SOURCE CODE

```
#-----  
#Sentiment Analyzer  
#-----  
  
from transformers import AutoTokenizer, AutoModelForSequenceClassification  
from transformers import pipeline  
import torch  
import logging  
from typing import Dict, List  
from ..core.config import settings  
logger = logging.getLogger(__name__)  
class SentimentAnalyzer:  
    def __init__(self):  
        self.model_name = settings.SENTIMENT_MODEL  
        self.device = 0 if torch.cuda.is_available() else -1  
        self.pipeline = None  
        self._load_model()  
    def _load_model(self):  
        """Load the sentiment analysis model"""  
        try:  
            self.pipeline = pipeline(  
                "sentiment-analysis",  
                model=self.model_name,  
                tokenizer=self.model_name,  
                device=self.device  
            )  
            logger.info(f"Loaded sentiment model: {self.model_name}")  
        except Exception as e:  
            logger.error(f"Error loading sentiment model: {e}")  
            # Fallback to a simpler model  
            try:  
                self.pipeline = pipeline(  
                    "sentiment-analysis",  
                    model="distilbert-base-uncased-finetuned-sst-2-english",  
                    device=self.device  
                )
```

```

        logger.info("Loaded fallback sentiment model")
    except Exception as e2:
        logger.error(f"Error loading fallback model: {e2}")
        self.pipeline = None
def analyze_sentiment(self, text: str) -> Dict:
    """Analyze sentiment of given text"""
    if not self.pipeline:
        return {
            "label": "neutral",
            "score": 0.5,
            "confidence": 0.0
        }
    try:
        # Truncate text if too long
        max_length = 512
        if len(text) > max_length:
            text = text[:max_length]
        result = self.pipeline(text)
        # Normalize score to -1 to 1 range
        label = result["label"].lower()
        score = result["score"]
        if label in ["positive", "pos", "label_2"]:
            normalized_score = score
        elif label in ["negative", "neg", "label_0"]:
            normalized_score = -score
        else: # neutral
            normalized_score = 0
        return {
            "label": "positive" if normalized_score > 0.1 else "negative" if normalized_score < -
0.1 else "neutral",
            "score": normalized_score,
            "confidence": score
        }
    except Exception as e:
        logger.error(f"Error analyzing sentiment: {e}")
        return {

```

```

        "label": "neutral",
        "score": 0.0,
        "confidence": 0.0
    }

def batch_analyze(self, texts: List[str]) -> List[Dict]:
    """Analyze sentiment for multiple texts"""
    results = []
    for text in texts:
        result = self.analyze_sentiment(text)
        results.append(result)
    return results

def get_sentiment_summary(self, articles: List[Dict]) -> Dict:
    """Get sentiment summary for multiple articles"""
    if not articles:
        return {
            "total_articles": 0,
            "positive_count": 0,
            "negative_count": 0,
            "neutral_count": 0,
            "average_score": 0.0
        }
    sentiments = []
    positive_count = 0
    negative_count = 0
    neutral_count = 0
    total_score = 0.0
    for article in articles:
        sentiment = self.analyze_sentiment(article.get("content", ""))
        sentiments.append(sentiment)
        label = sentiment["label"]
        score = sentiment["score"]
        if label == "positive":
            positive_count += 1
        elif label == "negative":
            negative_count += 1
        else:

```



```

        neutral_count += 1
    total_score += score
    return {
        "total_articles": len(articles),
        "positive_count": positive_count,
        "negative_count": negative_count,
        "neutral_count": neutral_count,
        "average_score": total_score / len(articles) if articles else 0.0,
        "sentiment_distribution": {
            "positive": positive_count / len(articles) * 100 if articles else 0,
            "negative": negative_count / len(articles) * 100 if articles else 0,
            "neutral": neutral_count / len(articles) * 100 if articles else 0
        }
    }
}

#-----
#Text Summarizer
#-----

from transformers import AutoTokenizer, AutoModelForSeq2SeqLM, pipeline
import torch
import logging
from typing import Dict, List
from ..core.config import settings
logger = logging.getLogger(__name__)
class TextSummarizer:
    def __init__(self):
        self.model_name = settings.SUMMARIZATION_MODEL
        self.device = 0 if torch.cuda.is_available() else -1
        self.pipeline = None
        self._load_model()
    def _load_model(self):
        """Load the summarization model"""
        try:
            self.pipeline = pipeline(
                "summarization",
                model=self.model_name,
                tokenizer=self.model_name,

```

```

        device=self.device
    )
    logger.info(f"Loaded summarization model: {self.model_name}")
except Exception as e:
    logger.error(f"Error loading summarization model: {e}")
    # Fallback to a smaller model
    try:
        self.pipeline = pipeline(
            "summarization",
            model="t5-small",
            device=self.device
        )
        logger.info("Loaded fallback summarization model")
    except Exception as e2:
        logger.error(f"Error loading fallback model: {e2}")
        self.pipeline = None
def summarize_text(self, text: str, max_length: int = 150,
                    min_length: int = 50) -> Dict:
    """Summarize the given text"""
    if not self.pipeline:
        return {
            "summary": text[:200] + "..." if len(text) > 200 else text,
            "confidence": 0.0,
            "error": "Model not available"
        }
    try:
        # Truncate text if too long for the model
        max_input_length = 1024
        if len(text) > max_input_length:
            text = text[:max_input_length]
        # Ensure text is not too short
        if len(text.strip()) < 50:
            return {
                "summary": text,
                "confidence": 1.0,
                "error": None
            }

```

```

    }
    result = self.pipeline(
        text,
        max_length=max_length,
        min_length=min_length,
        do_sample=False
    )
    return {
        "summary": result[0]["summary_text"],
        "confidence": 0.8, # BART doesn't provide confidence scores
        "error": None
    }
except Exception as e:
    logger.error(f'Error summarizing text: {e}')
    return {
        "summary": text[:200] + "..." if len(text) > 200 else text,
        "confidence": 0.0,
        "error": str(e)
    }

def batch_summarize(self, texts: List[str], max_length: int = 150) -> List[Dict]:
    """Summarize multiple texts"""
    results = []
    for text in texts:
        result = self.summarize_text(text, max_length)
        results.append(result)
    return results

def extract_key_points(self, text: str) -> List[str]:
    """Extract key points from text using sentence importance"""
    try:
        # Simple extractive summarization
        sentences = text.split('.')
        sentences = [s.strip() for s in sentences if len(s.strip()) > 20]
        # Score sentences by length and keywords
        scored_sentences = []
        for i, sentence in enumerate(sentences):
            score = len(sentence.split()) # Simple scoring based on word count

```

```

        scored_sentences.append((score, sentence, i))
    # Sort by score and take top sentences
    scored_sentences.sort(reverse=True)
    key_points = [s[1] for s in scored_sentences[:3]]
    return key_points
except Exception as e:
    logger.error(f'Error extracting key points: {e}')
    return []

```

5. CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

The EquityLens project effectively demonstrates how NLP and Generative AI techniques can be used to analyze public opinions and financial data. It provides accurate sentiment classification to help understand real-time financial trends and market behavior. The model is simple and efficient in its execution.

5.2 Who Can Use This Model?

User Type	Use
Investors	Analyze market sentiment for stock decision-making
Financial Analysts	Summarize long reports and extract key entities
Companies	Track brand reputation and competitor news
Students	Learn NLP and Generative AI concepts in finance

Table 5.2.1: Who Can Use This Model?

The above Table 5.2.1 identifies the target audience for the EquityLens tool, such as investors and analysts, and how they benefit from the AI insights.

5.3 Limitations

- **Sarcasm Detection:** The model cannot accurately detect sarcasm in financial commentary.
- **Context Understanding:** There is limited understanding of complex market contexts.
- **Data Quality:** The model's performance depends heavily on the quality of the data set used.

5.4 Future Improvements

- Advanced Architectures: Transitioning to Deep Learning models like LSTM or BERT.
- Real-time Integration: Adding live financial news API integrations.
- Language Support: Expanding the system to support multiple languages for global analysis.